GIDFINE LOAN PLATFORM

PHASE 4A: FINANCIAL LEDGER SCHEMA & IDEMPOTENCY AUDIT REPORT (REVISED)

Database Foundation & Maker-Checker Architecture | Branch: development | HEAD: 931411f

Date: October 21, 2026 (Correction Pass)	**Target Branch:** development (HEAD @ 931411f)
Starting Pre-Flight: Verified clean, ahead 5 commits	**main Branch:** Untouched (at f7ed8d1)
Scope: Schema, Indexes, Types, Tests (No Business Logic)	**Validation Status:** 100% Passing (44 Node + 6 Py + Vite)

1. Executive Summary

Phase 4A establishes the core database schema, compound indexing, foundational data types, and idempotency mechanics for the financial ledger and disbursement request system. In strict adherence to safety requirements, no financial business mutations, payment execution, or loan activation logic were implemented. All monetary values are strictly modeled in integer GHS minor units (pesewas), and segregation of duties (Maker-Checker rule: maker cannot approve own request) is built into the foundational data model.

2. Schema Tables & Compound Index Definitions

Table Name	Key Fields & Types	Compound Indexes
`disbursementRequests`	• agencyId: id('agencies') • applicationId: id('applications') • amountPesewas: integer (GHS pesewas) • currency: literal('GHS') • status: PENDING_APPROVAL | APPROVED | REJECTED | PROCESSED | CANCELLED • makerId: id('users') • checkerId: optional id('users') (checker != maker) • idempotencyKey: string (agency-scoped) • channel: optional (MOMO | BANK_TRANSFER | CHEQUE | CASH)	• `by_agency: [agencyId]` • `by_application: [applicationId]` • `by_agency_and_status: [agencyId, status]` • `by_agency_and_idempotency: [agencyId, idempotencyKey]` • `by_maker: [makerId]`
`ledgerEntries`	• agencyId: id('agencies') • applicationId: id('applications') • entryType: DISBURSEMENT | REPAYMENT_PRINCIPAL | REPAYMENT_INTEREST | REPAYMENT_FEE | REVERSAL | ADJUSTMENT • amountPesewas: integer (> 0) • currency: literal('GHS') • actorId: id('users') • disbursementRequestId: optional id • reversedEntryId: optional id (reversal link) • isReversal: boolean • externalReference: optional string • **idempotencyKey**: optional string (agency-scoped) • notes: optional string	• `by_agency: [agencyId]` • `by_application: [applicationId]` • `by_disbursement_request: [disbursementRequestId]` • `by_agency_and_idempotency: [agencyId, idempotencyKey]` • `by_reversed_entry: [reversedEntryId]`

3. Idempotency & Concurrency Guarantees vs Implementation Dependencies

To prevent confusion between database guarantees and application logic, the concurrency model is defined across three distinct tiers:

1. **Schema & Index Behavior:** Convex v1.47.0 table indexes (such as by_agency_and_idempotency) provide fast, structured B-tree lookup paths but **do not enforce unique constraints** at the storage layer. Missing or undefined idempotency keys are indexed as undefined.
2. **Documented Convex OCC Guarantees:** Convex mutations run with serializable ACID isolation via Optimistic Concurrency Control (OCC). When a mutation executes an index query, Convex tracks the read interval. A concurrent write into that interval invalidates the read set, causing an automatic transaction rollback and retry. This prevents phantom duplicates under concurrent requests.
3. **Guarantees Depending on Phase 4B Implementation:** OCC only protects transactions if the mutation explicitly executes the indexed lookup before insertion and enforces payload matching. Specifically, Phase 4B mutations must: a) guard against querying undefined idempotency keys, b) compare incoming amounts and application IDs against existing records via evaluateIdempotencyPayloadMatch, and c) throw an error if an idempotency key is reused with mismatched parameters.

4. Maker-Checker Segregation Status

In Phase 4A, the segregation rule (makerId !== checkerId) is implemented as a pure validation helper (validateMakerCheckerSegregation in ledgerTypes.ts) and covered in unit tests. **No approval mutations were introduced in Phase 4A.** Full server-side enforcement—extracting caller identity strictly from ctx.auth.getUserIdentity() and asserting maker inequality—remains a mandatory requirement for Phase 4B.

5. Verification & Validation Execution Results

Target Test Suite	Command Line	Exit Code	Status
TypeScript Static Analysis	`npx tsc --noEmit`	0	**PASSED (0 errors)**
Node Test Runner (All Suites)	`node --experimental-strip-types --test tests/*.test.ts`	0	**PASSED (44 / 44 tests)**
Phase 4A Focused Suite	`node --test tests/ledgerSchema.test.ts`	0	**PASSED (14 / 14 tests)**
Python Backend Tests	`pytest tests/ -q`	0	**PASSED (6 / 6 tests)**
Production Vite Bundle	`npx vite build`	0	**PASSED (1,845 modules)**

6. Strict Scope & Boundaries Confirmation

• **Branch Integrity:** Remaining strictly on development. main is untouched at f7ed8d1.

• **Change Control:** Zero commits created in Phase 4A. Zero pushes executed. Convex Cloud remains undeployed.

• **Business Logic Boundary:** Zero disbursement, approval, ledger posting, or repayment mutations implemented.

• **Next Phase Boundary:** Stopped after Phase 4A correction pass. Awaiting explicit authorization before proceeding to Phase 4B.